EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 08 — 05/11/2025

Author: Jonathan Dutoit

Professor: Maria Brbic

Applied Machine Learning

Classification pipeline

$\oplus \Rightarrow$ Data collection $\Rightarrow$ Model Selection $\Rightarrow$ Model assessment $\Rightarrow \oplus$

Data collection

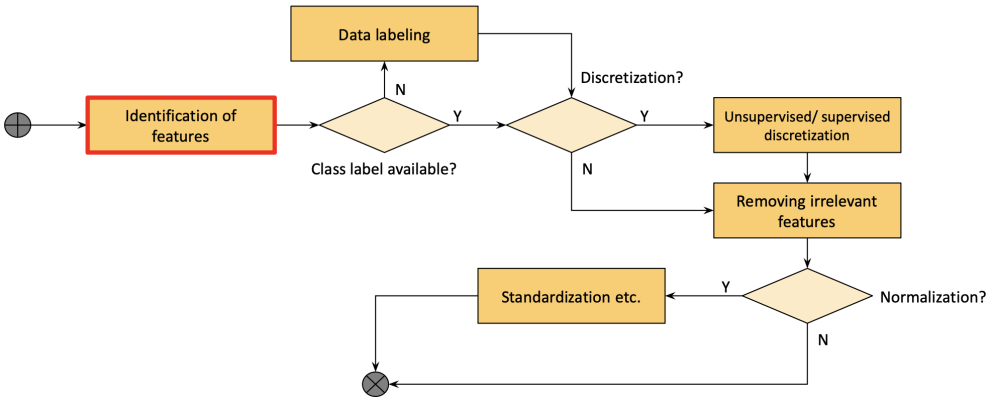
The first step is collecting data related to the classification task.

- Definition of the attributes (or features) that describe a data item and the class label.

Domain knowledge is needed.

What if assigning the class label would be too time-consuming or even impossible?

- Unsupervised methods (e.g., clustering)



Features

Different types of features

- Continuous (e.g., height, temperature, ...)
- Ordinal (e.g., ”agree”, ”don’t care”, ”disagree”, ...)

- Categorical (e.g., country, gender, ...)

New features can be generated from simple stats

- ***Feature engineering*** is considered a form of art, therefore it is often useful to find what other people did for similar problems

Some classifiers require categorical features ⇒ discretization

Labels

Collecting a lot of data (features) is often easy. Labeling data is time consuming, difficult, and sometimes even possible. Often *human expertise* is needed.

Potential labelers

- You
- Older days:
  - Undergraduate students
  - Domain experts ($$$)
- Now: crowdsourcing
  - Can get both amateurs (∼ undergrad students) and experts



Discretization

Why?

- Some classifiers want discrete features (e.g., simplets kinds of decision trees)
- Discrete features let a linear classifier learn non-linear decision boundaries
- Certain features selection methods require discrete (or even binary) features

Unsupervised: we don't want to use class labels for discretization

- Equal width
  - Divide the range into a predefined number of bins (bad for skewed data, e.g., from a power law)
- Equal frequency
  - Divide the range into a predefined number of bins so that every interval contains the same number of values. Doesn't take into account natural grouping of features.
- Clustering

Supervised: we use the class labels in order to perform discretization

- Start with very fine-grained discretization
- Test the hypothesis that membership in two adjacent intervals of a feature is independent of the class
- If they are independent, they should be merge
- Otherwise they should remain separate
- Independence test: χ^2 test ("chi-squared test")
- Continue recursively

Removing irrelevant features: Feature selection

- **Goal**: reduce set of N features to a subset of the best size $M < N$
- **Why?**
  - More interpretability
  - Less danger of overfitting
  - More efficient training
- **Problem**: There are 2^N possible subsets
- **Solutions**
  - Filtering as preprocessing ("offline")
  - Iterative feature selection ("online")

Offline feature selection

Rank features according to their individual predictive power; then select the best ones

- **Pros**
  - Independent of the classifiers (performed only once)
- **Cons**
  - Independent of the classifiers (ignore interaction with the classifier)
  - Assumes features are independent

Ranking of features

Continuous features (and ideally labels):

- Pearson's correlation coefficient (capturing strength of linear [!] dependence)

$$r = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^{n}(X_i - \bar{X})^2}\sqrt{\sum_{i=1}^{n}(Y_i - \bar{Y})^2}}.$$

Categorical features and labels:

- Mutual information (goes beyond linear dependence)

$$I(F;C) = H(C) - H(C \mid F) = H(F) + H(C) - H(F, C)$$
$$H(F) = -\sum_i P(f_i)\log_2 P(f_i)$$
$$H(F, C) = -\sum_i \sum_j P(f_i, c_j)\log_2 P(f_i, c_j)$$

- χ^2 method ("chi-squared")
  - Similar to χ^2 method for feature discretization
  - Test whether the feature is independent of label
  - Different w.r.t. mutual information: the χ^2 test checks the independence of the class and the feature, without indicating the strength or direction of any existing relationship (you just get a significance, a.k.a *p-value*)
- **Beware**: collectively relevant features may look individually irrelevant!

Online feature selection

Forward feature selection: greedily add features; evaluate on validation dataset; stop when no improvement.

- **Pros**
  - Interact with the classifier
  - No feature-independence assumption
- **Cons**
  - Computationally intensive

Backward feature selection: greedily remove features; evaluate on validation dataset; stop when no improvement

- **Pros**
  - Interact with the classifier
  - No feature-independence assumption
- **Cons**
  - Computationally intensive

Feature normalization

- Some classifiers can't easily handle features with different scales, e.g.,
  - Revenue in CHF: 10,000,000
  - of employees: 300
- Features with large values dominate the others, and the classifiers tends to over-optimize for them
- Even single feature may span many orders of magnitude
  - e.g., city size (most cities small, some huge)
  - Too little resolutions where data is dense, too much resolution where data is sparse

Logarithmic scaling

$$x_i^{'} = \log(x_i).$$

- Consider order of magnitude, rather than direct value
- Good for heavy-tailed features (e.g, from power laws)

Min-max scaling

$$x_i^{'} = (x_i - m_i)/(M_i - m_i).$$

where M_i and m_i are the max and min values of feature x_i respectively. The new feature $x_i^{'}$ lies in the interval [0, 1]

Standardization

$$x_i^{'} = (x_i - \mu_i)/\sigma_i.$$

where μ_i is the mean value of feature x_i, and σ_i is the standard deviation. The new feature $x_i^{'}$ has mean 0 and standard deviation 1

Dangers of standardization and scaling

Standardization

- Assume that the data has been generated by Gaussian
- Users mean and std $\Rightarrow$ not meaningful for heavy-tailed data (may be mitigated by log-scaling)

Min-max scaling

- If the data has outliers, they scale the typical values to a very small interval

Model selection

Model selection: high level

Need to choose type of model

- k-NN?
- Decision trees?
- random forest?

- Boosted decision trees?
- Logistic regression?
- Deep learning?

Model selection: low level

Usually a classifier has some "hyperparameters" to be tuned

- Set of features to include
- Distance function (e.g., k-NN)
- Number of neighbors (e.g., k-NN)
- Number of trees (e.g., random forest)
- Decision threshold (e.g., logistic regression)
- Regularization parameter
- Learning rate (for gradient descent algorithms)

Loss function

- Categorical output
  - e.g., 0-1 loss function, risk (= 1 minus accuracy):

$$J = \frac{1}{n}\sum_{i=1}^{n} \#(y \neq f(x_i)).$$

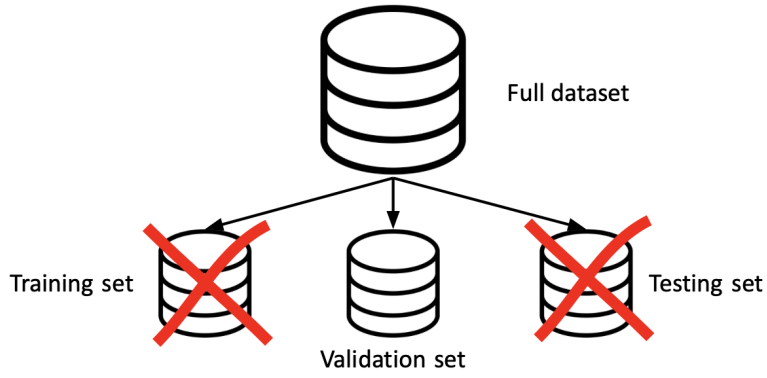
- Real-valued output
  - e.g., squared error:

$$J = \frac{1}{n}\sum_{i=1}^{n} (y_i - f(x_i))^2.$$

  - e.g., absolute error:

$$J = \frac{1}{n}\sum_{i=1}^{n} \mid y_i - f(x_i) \mid .$$

On what data to evaluate?

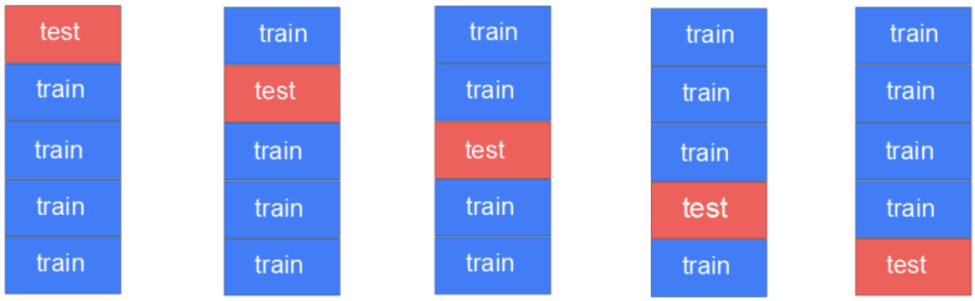


The validation set helps us to pick the right model!

What if you can't afford a 3-way split because you have too little data?

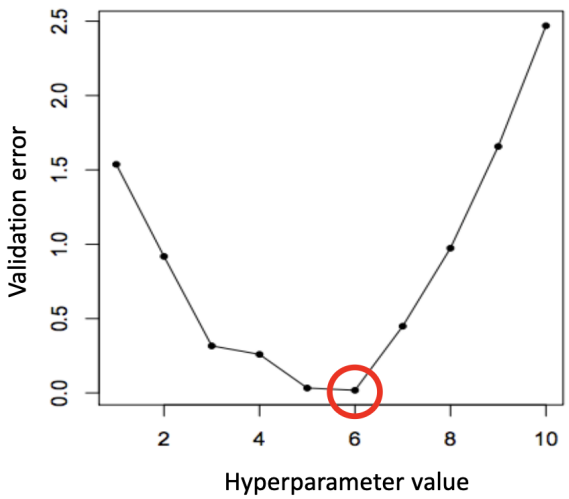
⇒ **Cross-validation**

Cross-validation



- Last lecture: leave-one-out cross-validation == N-fold cross-validation (where N is the #data points). This is quite computationally expensive.
- More efficient: m-fold cross-validation (in above picture: m=5)
- Average performance over the m red portions ⇒ validation error
- Repeat procedure for all candidate models and pick the one with the lowest validation error

Example: number of neighbors in k-NN



Performance metrics for binary classification

For categorical binary classification, the usual metrics are based on the confusion matrix, which has 4 values:

- True Positives (positive examples classified as positive)
- True Negatives (negative examples classified as negative)
- False positives (negative examples classified as positive)
- False Negatives (positive examples classified as negative)

		Class	
		Pos	Neg
Classified	Pos	TP	FP
	Neg	FN	TN

Accuracy

$$A = \frac{TP+TN}{TP+TN+FP+FN} = \frac{TP+TN}{N}$$

Appropriate metric when

- classes are not skewed
- errors (FP, FN) have the same importance

Accuracy (skewed example)

Classifier 1		Class	
		Fraud	¬Fraud
Classified	Fraud	5	10
	¬Fraud	5	80

Accuracy = 85/100 = 85%

Always ¬Fraud		Class	
		Fraud	¬Fraud
Classified	Fraud	0	0
	¬Fraud	10	90

Accuracy = 90/100 = 90%

The second classifier always says no but has higher accuracy...

Precision

What fraction of positive predictions are actually positive?

$$\mathrm{P} = \frac{TP}{TP+FP}$$

Recall

What fraction of actually positive examples did I recognize as such?

$$\mathrm{R} = \frac{TP}{TP+FN}$$

Classifier 1

Classified	Class: Cancer	Class: ¬Cancer
Cancer	45	20
¬Cancer	5	30

Classifier 2

Classified	Class: Cancer	Class: ¬Cancer
Cancer	40	10
¬Cancer	10	40

- Accuracy: 0.75
- Precision: 0.69
- Recall: 0.9

- Accuracy: 0.8
- Precision: 0.8
- Recall: 0.8

In this case, **Recall** is more important because we want to recognize those who truly have cancer! But we should always take into account the trade-off between Recall and Precision!

F-score

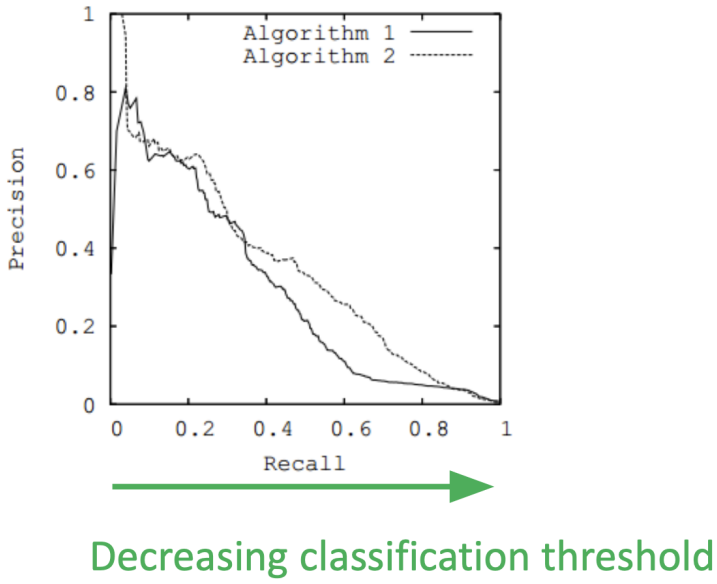
Sometime it's necessary to have a single metric to compare classifiers.

F-score (or F1-score): harmonic mean of precision and recall:

$$F1 = 2 \cdot \frac{P \cdot R}{P + R}$$

Precision and recall can be **differently weighted**, if one is more important than the other.

Precision/Recall curve



In this case we would chose algorithm 2 since it almost always a better performance.

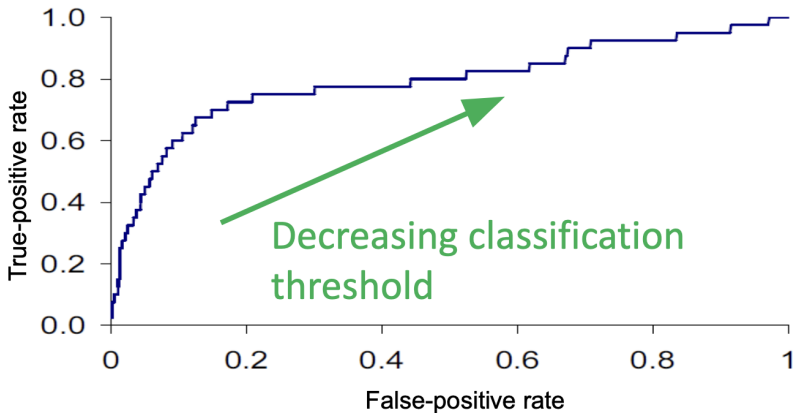
Precision/recall curve is very useful tool to compare models. There's no need to pre-select the decision threshold.

ROC curve

ROC = Receive-Operating Characteristic

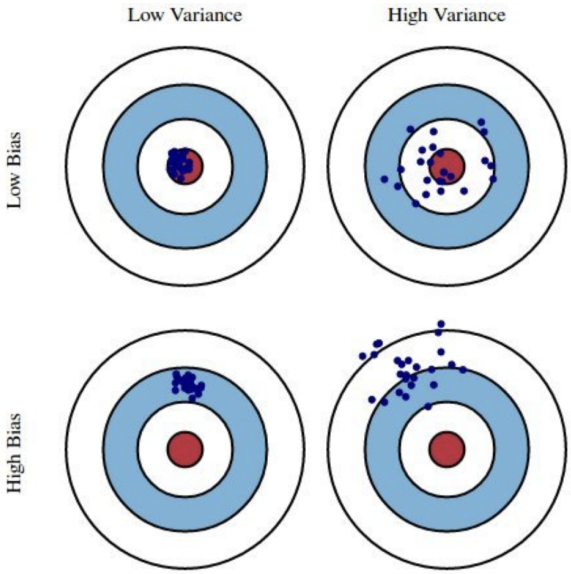
Y-axis: true-positive rate = TP/(TP + FN) a.k.a recall

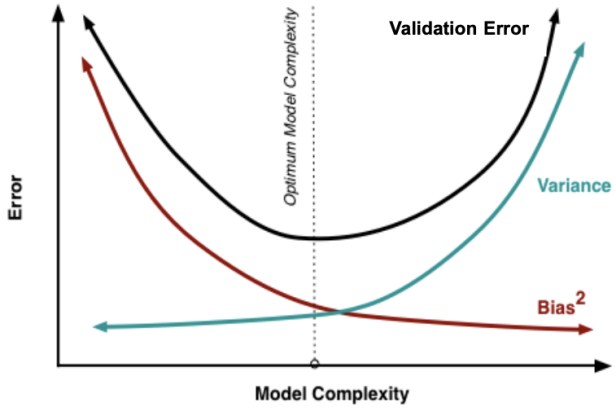
X-axis: false-positve rate = FP/(FP + TN)

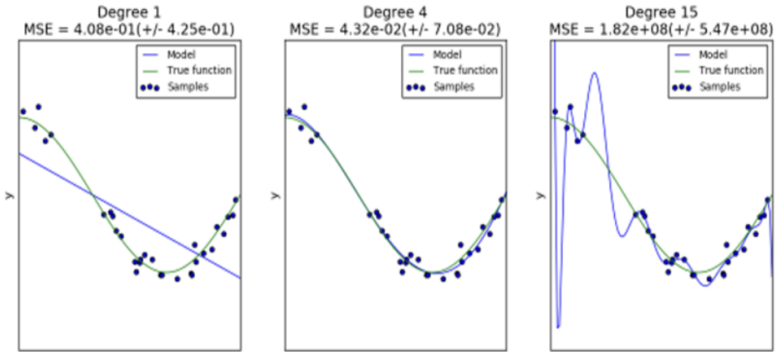


ROC AUC is the "area under the curse"- a single number that captures the overall quality of the classifier. IT should be between 0.5 (random classifier) and 1.0 (perfect).

Bias and variance

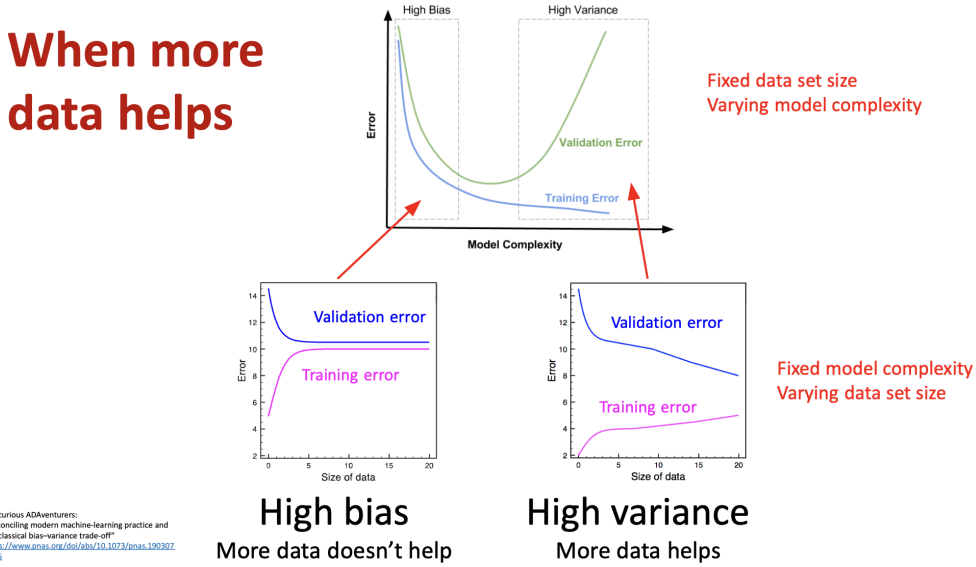






Often we don't have the variance and bias curves... So we use the validation error and training error metrics!

Thus, bias and variance ban be assessed by comparing the error metric on the training set and the validation set ⇒ always **plot learning curves** (training set size vs. training/validation errors)



Model assessment

- Model assessment is the goal of estimating the performance of a fixed model (i.e, the best model found during model selection)
- Ideally under real-world conditions
- Use held-out test set that you've never seen during training